

CareerPilot — Architecture Review

Senior AI-systems-engineer / hackathon-judge pass on the Anakin Forge 2026 spec

The core idea is strong: a closed loop of **resume** → **live jobs** → **personalized reasoning** → **skill gap** → **preparation** → **interview** → **next action**, with visible agent activity and explainable scores. That loop, polished, is a genuinely competitive submission. The problem is scope: as written, this is roughly a 4–6 week product specification labeled as a one-week MVP. Eight named agents, LangGraph orchestration, a dual Next.js + FastAPI stack, pgvector embeddings, browser automation, and an adaptive audio interview is more surface area than a small team can build *and* demo reliably in seven days. The recommendations below are organized by priority, followed by direct answers to the ten review questions posed in the spec.

MUST FIX

- **Cut agent count from 8 to ~4 real agents for MVP.** Merge International Eligibility into the Job Agent's extraction step (a field on the job record, not a standalone agent with its own state). Merge Skill Gap into the Fit/Match step. Target: Resume Agent, Job+Fit Agent, Interview Agent, Career Strategist. Application Agent and Action Agent become thin UI-triggered functions, not persistent agents with their own reasoning loop.
- **Drop LangGraph unless the team already knows it well.** The demo path is almost entirely linear (resume → search → scrape → extract → match → gaps → interview → recommendation) with one or two branch points. A plain orchestrator function with if/elif routing gets identical demo behavior with far less framework debugging risk in week one.
- **Remove browser/Wire automation from MVP entirely.** This is the single highest-risk, most fragile piece (automating arbitrary application websites), and the spec's own DO-NOT-BUILD list already excludes automatic submission — so the Action Agent has no real job to do yet. Keep the 'prepare for approval' UI and hard-stub the execute step (e.g., mark as 'ready to apply' + open the application URL). This removes the single most likely live-demo failure point.
- **Sponsorship classification needs a hard rule, not just an LLM label.** Require the model to quote the exact source sentence it used as evidence; if no explicit sentence is found, the system defaults to UNCLEAR — never let the LLM output EXPLICIT on inference alone. This is a real-world-harm surface (international students making visa-relevant decisions), not just a UX nicety, and it's cheap to enforce with a quote-required schema field plus a keyword sanity check.
- **Build the cached/offline demo fallback in week one, not Phase 8.** The spec correctly identifies this need but sequences it last. If live search/scrape breaks during judging, there is no time to build a fallback on the spot. Seed 15–20 realistic cached job records on day one and demo against them by default, with live mode as a bonus toggle.

SHOULD FIX

- **Collapse the two-backend split if possible.** Next.js + a separate FastAPI service is two deployments, two auth boundaries, and two places for CORS/env bugs to hide, for a team with one week. If Python is only needed for the agent/LLM layer, consider Next.js API routes calling out to a single Python service, or just running everything through FastAPI with a lighter frontend. Only keep the split if it's already comfortable for the team.
- **Cut pgvector/embeddings unless they're actually load-bearing.** The fit-scoring section as specified is a weighted heuristic score (skills/education/experience/location/salary/sponsorship), not semantic similarity search. If embeddings aren't driving an actual feature, they're infrastructure weight with no demo payoff — drop them for MVP and revisit only if there's time left.
- **Simplify auth.** Supabase email/password plus RLS policies done correctly takes longer than it looks. A single seeded demo account (with real signup as a stretch goal) removes a whole class of last-minute bugs.
- **Cut the audio/'Recording' interview UI.** There's no real speech pipeline planned, and a fake mic icon reads worse to judges than a clean, fast text Q&A; interface. The adaptive follow-up behavior is the impressive part — keep that, drop the audio theater.
- **Keep the ATS rubric simple and heuristic, not pseudo-ML.** 5–6 transparent weighted checks (keyword overlap %, section completeness, quantified-bullet ratio, formatting flags) are both faster to build and more defensible under judge questioning than anything that looks like a trained model. The disclaimer language already in the spec is good — keep it.
- **Trim the formal test suite to smoke tests.** Full unit + integration + mocked-agent test coverage is good engineering practice but is very likely to get cut under time pressure anyway — plan for a handful of smoke tests on the extraction and scoring functions, not a full pyramid.

NICE TO HAVE (build only after the core loop works end to end)

- Agent activity trace / 'CareerPilot is working...' panel — genuinely strong demo device, worth the polish time once the pipeline is solid.
- Match-score breakdown / 'why not 100%?' explainability click — cheap to add once the weighted score exists, and it's one of the more memorable judge-facing moments.
- Cover letter and tailored-CV generation — good demo value, but sequence it after the interview flow, not before.
- Applications Kanban board and Career Plan timeline — nice polish, not core-loop critical.
- Skill-gap frequency insight ('AWS appears in 61% of your shortlisted jobs') — cheap once skill gaps exist per job, and it's a good demo beat.

DO NOT BUILD (confirming and extending the spec's own list)

- Full LinkedIn/universal browser automation, automatic application submission — already correctly excluded.
- Mobile app, fine-tuned models, custom vector DB infra, microservices, Kubernetes — already correctly excluded.
- Google OAuth — one login method is enough for a demo.

- A general-purpose 'Wire' action layer for arbitrary sites — scope it to zero for MVP, not 'one controlled workflow'; even one controlled workflow is new fragile surface for marginal demo value given the human-approval gate already tells the story without it actually executing.
- Notification systems, multi-tenant org support, admin panels — none of this is judge-visible in a 3–5 minute demo.

Answers to the Ten Review Questions

1. Is LangGraph justified?

Marginal. The demo path is close to linear with a couple of branch points (apply vs. improve). A hand-rolled orchestrator achieves the same visible behavior with much less framework-debugging risk in week one. Keep it only if someone on the team already has hands-on LangGraph experience; otherwise it's a week-one time sink for zero judge-visible benefit.

2. Are the agents correctly separated?

Conceptually yes, but there are too many of them for a small team on a one-week clock. Merge International Eligibility into the Job Agent's extraction output, and merge Skill Gap into the Fit/Match step. Real target: Resume Agent, Job+Fit Agent, Interview Agent, Career Strategist — four agents with clear boundaries beat eight agents with thin ones.

3. Is the orchestrator too complex?

As specified (LangGraph state machine with dynamic agent selection), yes. A simple linear pipeline with two or three conditional branches covers every scene in the demo script and is far easier to debug under time pressure.

4. Is the ATS scoring methodology defensible?

Yes, provided it stays a transparent weighted heuristic (keyword overlap, section completeness, quantified-bullet ratio, formatting checks) with the 'compatibility estimate, not a real ATS prediction' disclaimer already in the spec. Don't dress it up as anything more rigorous than that — the honesty is a feature, not a weakness, in front of judges.

5. Is sponsorship classification safe?

Safe in concept, risky in implementation. The five-way EXPLICIT/LIKELY/UNCLEAR/NO/VERIFY taxonomy is the right idea, but it only works if the system requires a quoted source sentence before it will output EXPLICIT or NO, and defaults to UNCLEAR otherwise. Without that guardrail, an LLM will confidently over-classify from weak evidence — which is a real-world harm surface given the audience, not just a demo-quality issue.

6. Is Anakin being used efficiently?

The Search → filter → Scrape pattern (roughly 24 credits per batch in the spec's own example) is the right shape, and correctly avoids Agentic Search as a default. Two additions: persist scraped jobs so repeat searches never re-scrape the same URL, and hard-cap scraped jobs per demo run (e.g. 15) so dev-time testing doesn't eat into the 300-credit budget before judging even starts.

7. Can this realistically be built in 7 days?

Not as specified. Eight agents, LangGraph, pgvector, a dual-service backend, an adaptive audio interview, browser automation, and a full applications Kanban board is a multi-week build. A realistic 7-day cut: Resume Agent, Job Agent (search + scrape + extract), Fit/skill-gap scoring, a text-based adaptive interview, a dashboard with a Next Best Action card, and the agent-activity trace. That set is fully demoable and leaves slack for the inevitable last-day bugs.

8. What would a hackathon judge consider impressive?

The closed loop itself, made visible: the live agent-activity feed, the sponsorship-evidence citation shown next to its source, the 'why 94% and not 100%' explainability click, and the skill-gap frequency insight. None of that requires eight agents or a sophisticated orchestration framework — it requires the four-agent loop working smoothly and looking considered.

9. What would make the demo fail?

In rough order of likelihood: live scraping/search failing or rate-limiting mid-demo with no cached fallback ready; Anakin credits exhausted from dev-time testing before judging; a browser-automation step timing out on-stage; LangGraph state bugs surfacing under a code path never hit in rehearsal; auth/DB flakiness from a rushed Supabase setup. The cached demo dataset and the browser-automation cut both directly address the top two risks.

10. What should be cut?

Browser/Wire automation, embeddings/pgvector (unless genuinely load-bearing), International Eligibility and Skill Gap as separate top-level agents, the audio interview UI, the full Applications Kanban tracker, cover-letter/application-material generation (demote to stretch goal), the formal test pyramid (keep smoke tests only), and Google OAuth.

Bottom line: keep the pitch — "CareerPilot decomposes a career goal into research, analysis, preparation and action tasks, reads live web data through Anakin, maintains state across specialized agents, and adapts its next step based on the result" — but build it with four disciplined agents and a linear orchestrator instead of eight and a state-machine framework. The judge-visible magic is the closed loop and its explainability, not the agent count or the infrastructure choices.